

Lab Test 1

[25 marks]**Coding time: 1.5 hours****General Instructions:**

1. You will perform the lab test on your personal laptop.
2. All forms of external communications on your laptop are not permitted during the test. Your invigilators will provide instructions on how to disable such communications before the start of the test.
3. You may refer to any file on your laptop during the test.
4. Make sure your code can generate exactly the same output as we show in the sample runs. You may be penalized for missing spaces, missing punctuation marks, misspellings, etc., in the output.
5. Do not hardcode. We will use different test cases to test and grade your solutions.
6. Follow standard Python coding conventions (e.g., naming functions and variables).
7. Python script files that cannot be executed will NOT be marked, hence, you will be awarded 0 marks. You may wish to comment out the parts in your code that cause execution errors.
8. Include your name as the author in the comments of all your submitted source files.
For example, include the following block of comments at the beginning of each source file you need to submit.

```
# Name: Lum Ting Wong  
# Email ID: lum.ting.wong.2024
```

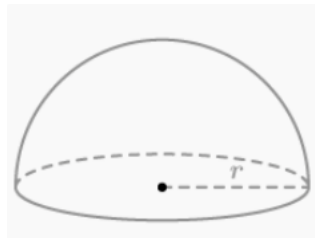
Instructions on how to submit your solutions:

1. Before the test begins, you will be instructed to download the resource files for the test from eLearn. After downloading the resource files, follow the invigilator's instructions and disable external communications on your laptop.
2. You will be given a password to unzip the downloaded .zip file. After unzipping the file, **you should rename the folder to your email id.**
For example, if your SMU email is lum.ting.wong.2024@scis.smu.edu.sg, you should rename the folder to **lum.ting.wong.2024**
You need to save your solutions in this folder for later submission. You may be penalized for not following our instructions.
3. When the test ends, you will be instructed to enable your laptop's internet and submit your solutions as a single zip file to eLearn. For example, if your folder is named lum.ting.wong.2024, you will be uploading lum.ting.wong.2024.zip

Question 1:**[8 marks]****Part (a) (Difficulty Level: *)**

Inside `q1a.py`, implement a function called `get_hemisphere_volume()`. This function takes in one parameter called `r` (type: `float`), a positive decimal number representing the radius of a hemisphere (a hemisphere is simply half a sphere).

The formula for the volume of a hemisphere is as follows:



$$V = \frac{2}{3}\pi r^3$$

Note that π is the mathematical **constant**. You can assume that the value of π is equal to 3.14

The function returns the volume of the hemisphere with a radius `r` according to the formula above. The return value is type: `float` rounded to at most two decimal places.

For example:

- `get_hemisphere_volume(10.5)` returns 2423.3 because $\frac{2}{3} \times \pi \times 10.5^3 = 2423.295$ and, after rounding to at most two decimal places, the result is 2423.3
- `get_hemisphere_volume(5.7)` returns 387.67 because $\frac{2}{3} \times \pi \times 5.7^3 = 387.67068$ and, after rounding to at most two decimal places, the result is 387.67

You can use the `round()` function to round a float number to two decimal places. For example: `round(3.567, 2)` will return 3.57

Use `q1a_test.py` to test your code. Do NOT modify `q1a_test.py`. Only `q1a.py` will be graded.

Part (b) (Difficulty Level: **)

Write a function to determine if a particular item is allowed in the carry-on luggage of an airplane based on the type of item.

Inside `q1b.py`, implement a function called `is_allowed_in_carryon()`. The function takes in two parameters:

- `item_category` (type `str`): The category of the item. This string is case-sensitive. For example, "liquids" is different from "Liquids".
- `quantity` (type `int`): The quantity of the item (depending on the category type). If the item category is "Liquids" then the quantity represents the number of millilitres (ml). Otherwise, the quantity represents the number of items.

Lab Test 1 – COR-IS1704

The function returns whether the item is allowed in carry-on luggage for the given quantity, as shown in the table below.

- The function returns a **Boolean value**, `True` or `False`, for the categories listed in the table below, depending on the item category and quantity.
- For any other category, the function returns the **string** : `"Contact Staff"`

Category of Item	Quantity	Is allowed in Carry-on Luggage
Electronics	Any	True
Liquids	More than 100 ml	False
	Less or equal to 100 ml	True
Weapons	Any	False
Food	Any	True
For any other Category	Any	"Contact Staff"

Do not hard-code the return value. Solutions of this nature will receive no marks.

Examples:

- `is_allowed_in_carryon("Electronics", 3)` returns `True`
- `is_allowed_in_carryon("Weapons", 1)` returns `False`
- `is_allowed_in_carryon("Liquids", 75)` returns `True`
- `is_allowed_in_carryon("Liquids", 120)` returns `False`
- `is_allowed_in_carryon("Food", 2)` returns `True`
- `is_allowed_in_carryon("Tools", 1)` returns the string `"Contact Staff"`
- `is_allowed_in_carryon("Aerosols", 1)` returns the string `"Contact Staff"`

Note: Make sure that the pre-defined categories, namely `"Electronics"`, `"Liquids"`, `"Weapons"`, and `"Food"` are spelled correctly in your code. The categories are case-sensitive, e.g., `"liquids"` is NOT the same as `"Liquids"`.

Use `q1b_test.py` to test your code. Do NOT modify `q1b_test.py`. Only `q1b.py` will be graded.

Question 2**[8 marks]****Part (a) (Difficulty Level: *)**

In **q2a.py**, implement a function called `count_short_strings()`. The function takes in the following parameters:

- `str_list` (type: `list`): This is a list of strings. You can assume that each element in this list is a string. It is possible for `str_list` to be empty.
- `n` (type: `int`): This is a number. You can assume that this is always a positive integer.

The function returns the number of strings in `str_list` that are **strictly** shorter than `n`. A string whose length is exactly `n` is NOT counted. If no string in `str_list` is shorter than `n` characters, the function returns 0.

For example, `count_short_strings(['abcd', '123', '**', '!@#$$%^'], 4)` returns 2 because among the strings in the list, only '123' and '**' are shorter than 4 characters.

Some other examples are shown below:

- `count_short_strings(['+++++', 'purple', 'pear', 'tiger'], 5)` returns 1 because only the string 'pear' contains less than 5 characters.
- `count_short_strings(['1234567890', 'abcdefg'], 1)` returns 0 because there are no strings in the list with less than 1 character.
- `count_short_strings([''], 1)` returns 1 because the empty string '' contains less than 1 character.
- `count_short_strings([], 1)` returns 0 because there is no string in the empty list.

Use **q2a_test.py** to test your code. Do NOT modify `q2a_test.py`. Only **q2a.py** will be graded.

Part (b) (Difficulty Level: **)

In `q2b.py`, implement a function called `get_longest_string()`. The function takes in a list of strings, `word_list`, and returns a tuple consisting of the longest string and the number of characters in that string. If there is more than one longest string, the function returns the one that appears later in the list. If the list is empty, the function returns `None`.

Example 1:

If the function is invoked like this:

```
print(get_longest_string(['grape', 'lime', 'pear']))
```

the following output is generated:

('grape', 5)

Example 2:

If the function is invoked like this:

```
print(get_longest_string(['grape', 'apple', 'pear']))
```

the following output is generated:

('apple', 5)

Note: 'grape' and 'apple' are both longest strings with only 5 characters. 'apple' is returned because it appears after 'grape'.

Example 3:

If the function is invoked like this:

```
print(get_longest_string(['', 'orange', 'durian']))
```

the following output is generated:

('durian', 6)

Example 4:

If the function is invoked like this:

```
print(get_longest_string(['']))
```

the following output is generated:

('', 0)

Example 5:

If the function is invoked like this:

```
print(get_longest_string([]))
```

the following output is generated:

None

Use `q2b_test.py` to test your code. Do NOT modify `q2b_test.py`. Only `q2b.py` will be graded.

Question 3:**[3.5 marks]****(Difficulty Level: **)**

The KK Hospital wants to implement a system that manages and analyzes patient records. You are tasked with writing a function that processes patient data and outputs relevant statistics.

In **q3.py**, define a function called `process_patient_data()`. The function takes in a parameter called `patients`, which is a list of tuples. Each tuple contains the following information about a patient:

- `name (type: str)`: The name of the patient.
- `age (type: int)`: The age of the patient.
- `gender (type: str)`: The gender of the patient, either "Male", "Female", or "Not specified".
- `ailments (type: list[str])`: A list of ailments or medical conditions the patient has. This list may be empty. Ailment strings are never empty.
- `discharged (type: bool)`: A Boolean value indicating if the patient has been discharged (`True`) or not (`False`).

The function should return a tuple that provides the following statistics in order:

- `average_age (type: float)`: The average age of all patients. Please use the `round()` function to round the result to at most two decimal places.
- `male_patients (type: int)`: The total number of male patients.
- `female_patients (type: int)`: The total number of female patients.
- `discharged_patients (type: int)`: The total number of discharged patients.
- `unique_ailments (type: list[str])`: A list of unique ailments in the patient data. The list should have the ailments in the same order as they show up in the patient data.

You may assume there is at least one patient on the list. The function should also meet the following requirements:

1. The function should handle cases where there are patients whose gender is "Not specified". These patients should not be counted as "male_patients" or "female_patients".
2. The function must handle special cases of the input data, e.g., no ailments or no female patients in the list. You may assume that there are no duplicate entries in the patient list or the associated list of ailments for that patient.
3. The ailments and gender strings are case-sensitive. For example, "Asthma" is different from "asthma".

Use **q3_test.py** to test your code. Do NOT modify `q3_test.py`. Only **q3.py** will be graded.

Example 1:

If the list of patients is:

```
patients = [  
    ("John Doe", 45, "Male", ["Diabetes"], True),  
    ("Jane Smith", 38, "Female", ["Hypertension", "Asthma"], False),  
    ("Sam Brown", 60, "Male", ["Asthma"], True),  
    ("Anna White", 29, "Female", ["Allergy"], True)  
]
```

The function should return:

```
(43.0, 2, 2, 3, ["Diabetes", "Hypertension", "Asthma", "Allergy"])
```

Lab Test 1 – COR-IS1704

Example 2:

If the list of patients is:

```
patients = [  
    ("Alice Blue", 40, "Female", ["Asthma"], True),  
    ("Bob Green", 55, "Male", ["Diabetes"], True),  
    ("Sam Lee", 30, "Male", ["Diabetes", "Hypertension", "Asthma"], True)  
]
```

The function should return:

```
(41.67, 2, 1, 3, ["Asthma", "Diabetes", "Hypertension"])
```

Example 3:

If the list of patients is:

```
patients = [  
    ("Alice Blue", 40, "Not Specified", [], True),  
    ("Bob Green", 55, "Not Specified", [], True),  
    ("Sam Lee", 30, "Male", ["Asthma"], False)  
]
```

The function should return:

```
(41.67, 1, 0, 2, ["Asthma"])
```

Question 4:**[5.5 marks]**

A unique algebraic notation records international chess matches as a string. In this problem, we will use a similar notation for the highly competitive game of international Tic-Tac-Toe.

Part (a) (Difficulty Level: *)**

In **q4a.py**, implement a function called `get_board_state()` that accepts two parameters.

- `N (type: int)`: The size of a square Tic-Tac-Toe board with `N` rows and `N` columns. $1 < N \leq 10$
- `game (type: str)`: A string that records a sequence of moves in a game. The game string may be empty, denoting a game with no moves.

A valid move places an `x` or `o` marker into an empty position. There may be at most one more `x` or `o` marker on the board, as the players take turns.

Each move in the string has the form `row,col,player`, where `row` is a row index, `col` is a column index, and `player` is either `x` or `o`. Both `row` and `col` are integers in the range `[0-9]`.

- See examples for clarification.

The function **returns** a list of strings (type: `list[str]`) that represent the state of the game board.

- The list contains `N` strings. Each string defines the state of one row in the board.
- Each string is `N` characters long.
- Each position on the board can be:
 - `"-"` (minus): Empty
 - `"x"` (lower-case): Occupied by `x`
 - `"o"` (lower case): Occupied by `o`

Note: The `game` string is always valid. We will never give strings where `x` and `o` occupy the same space, or coordinates are outside the board, etc.

Use **q4a_test.py** to test your code. Do NOT modify `q4a_test.py`. Only **q4a.py** will be graded.

Example:

```
game = '11x20o22x10o00x'
board = get_board_state(3, game)
print('Your output:', board)
```

Your output: `['x--', 'ox-', 'o-x']`

Based on the output list, `q4a_test.py` renders the following 2D game board:

```
  0 1 2
0 x - -
1 o x -
2 o - x
```

In this game,

- Move 1, `x` takes cell (1, 1) and is derived from `game = '11x20o22x10o00x'`
- Move 2, `o` takes cell (2, 0) and is derived from `game = '11x20o22x10o00x'`
- Move 3, `x` takes cell (2, 2) and is derived from `game = '11x20o22x10o00x'`
- Move 4, `o` takes cell (1, 0) and is derived from `game = '11x20o22x10o00x'`
- Move 5, `x` takes cell (0, 0) and is derived from `game = '11x20o22x10o00x'`

Lab Test 1 – COR-IS1704

Part (b) (Difficulty Level: *)**

Connoisseurs of Tic-Tac-Toe know that gravity makes the game more fun. You may know it better as “Connect-Four.” In this version of the game, markers (i.e., either x or o) placed above a column (above row 0) fall to the “lowest” unoccupied cell.

In **q4b.py**, write a function `get_board_state_gravity()` with the same inputs and outputs as in Part (a), *except* the `game` string contains a single coordinate for each move, denoting the column in which a marker is placed.

Your function must determine what row the marker comes to rest and update the board accordingly.

In the example below, we have rendered the 2D game board after each move to help you visualize the process. Your output should be a list of strings that represent the **last** state.

game = '0x1o1x1o0x2o'																							
Move 1 : 0x				Move 2 : 1o				Move 3 : 1x				Move 4 : 1o				Move 5 : 0x				Move 6 : 2o			
0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
0	-	-	-	0	-	-	-	0	-	-	-	0	-	-	-	0	-	-	-	0	-	-	-
1	-	-	-	1	-	-	-	1	-	-	-	1	-	o	-	1	-	o	-	1	-	o	-
2	-	-	-	2	-	-	-	2	-	x	-	2	-	x	-	2	x	x	-	2	x	x	-
3	x	-	-	3	x	o	-	3	x	o	-	3	x	o	-	3	x	o	-	3	x	o	o

- Move 1, an x piece placed in column 0 falls to row 3.
- Move 2, an o piece placed in column 1 falls to row 3.
- Move 3, an x piece placed in column 1 falls to row 2.
- Move 4, an o piece placed in column 1 falls to row 1.
- Move 5, an x piece placed in column 0 falls to row 2.
- Move 6, an o piece placed in column 2 falls to row 3.

You may assume that the inputs are always valid. For example, you will never place a piece into a full column.

Use **q4b_test.py** to test your code. Do NOT modify `q4b_test.py`. Only **q4b.py** will be graded.

Example:

```
game = '0x1o1x1o0x2o'
board = get_board_state_gravity(4, game)
print('Your output:', board)
```

Your output: ['----', '-o--', 'xx--', 'xoo-']

Based on the output list, `q4b_test.py` renders the following 2D game board:

```
 0 1 2 3
0 - - - -
1 - o - -
2 x x - -
3 x o o -
```

Part (c) (Difficulty Level: *)**

In **q4c.py**, write a function called `get_who_won()` to determine who won a game.

The function takes two parameters:

- `M (type: int)`: The number of consecutive `x` or `o` markers in a row, column, or diagonal needed to win, where $M > 1$.
- `board (type: list[str])`: The output of either `get_board_state()` or `get_board_state_gravity()`, from Part (a) or Part (b), respectively, representing the positions `x` and `o` occupy at some point during a game.

The function returns:

- `None (type: NoneType)`: There is no winner, and there are empty board positions.
- `"tie" (type: str)`: There is no winner, and all board positions are occupied.
- `"x" (type: str)`: `x` has `M` consecutive markers in a row, column, or diagonal.
- `"o" (type: str)`: `o` has `M` consecutive markers in a row, column, or diagonal.

Do not hard-code or randomly guess the output. Solutions of this nature will receive no marks.

You may assume that the inputs are always valid and there are no games where both `x` and `o` win.

Use **q4c_test.py** to test your code. Do NOT modify `q4c_test.py`. Only **q4c.py** will be graded.

Example 1:

```
board = ['-----', '-----', 'x-----', 'xo----', 'xo----']
print('Who won:', get_who_won(3, board))
```

```
Who won: x
```

`q4c_test.py` renders the following 2D game board:

```
  0 1 2 3 4
0 - - - - -
1 - - - - -
2 x - - - -
3 x o - - -
4 x o - - -
```

In this game, `x` won because `x` has `M` consecutive pieces in column 0 (where $M = 3$).

Example 2:

```
board = ['-----', '-----', '-----', 'xo----', 'xo----']
print('Who won:', get_who_won(3, board))
```

```
Who won: None
```

`q4c_test.py` renders the following 2D game board:

```
  0 1 2 3 4
0 - - - - -
1 - - - - -
2 - - - - -
3 x o - - -
4 x o - - -
```

Lab Test 1 – COR-IS1704

In this game, no one won because neither x nor o has M consecutive pieces in a row, column, or diagonal (where $M = 3$).

Example 3:

```
board = ['xxo', 'oox', 'o-x']
print('Who won:', get_who_won(3, board))
```

Who won: o

q4c_test.py renders the following 2D game board:

```
  0 1 2
0 x x o
1 o o x
2 o - x
```

In this game, o won because o has M consecutive pieces diagonally (where $M = 3$).

Example 4:

```
board = ['xxo', 'oox', 'xxo']
print('Who won:', get_who_won(3, board))
```

Who won: tie

q4c_test.py renders the following 2D game board:

```
  0 1 2
0 x x o
1 o o x
2 x x o
```

This game is a tie because neither x nor o has M consecutive pieces in a row, column, or diagonal (where $M = 3$) and no further moves are possible.

Example 5:

```
board = ['----', '---o', '-xo-', '-ox-']
print('Who won:', get_who_won(3, board))
```

Who won: o

q4c_test.py renders the following 2D game board:

```
  0 1 2 3
0 - - - -
1 - - - o
2 - x o -
3 - o x -
```

In this game, o won because o has M consecutive pieces diagonally (where $M = 3$).

*** END OF PAPER ***